第 9 节 实操任务 2：实现 Supervisor 监督...

> **学习目标**：`python patterns/supervisor.py` 能运行审核循环，最多 3 轮
> 目标文件：`patterns/supervisor.py`

步骤 1：用 AI 编程工具生成 supervisor.py

> 以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。

提示词：

```
请帮我编写 patterns/supervisor.py，实现 Supervisor 监督模式：

需求：
1. Worker Agent：接收任务，输出 JSON 格式的分析报告
2. Supervisor Agent：对 Worker 的输出进行质量审核
   - 评分维度：准确性(1-10)、深度(1-10)、格式(1-10)
   - 输出 JSON: {"passed": bool, "score": int, "feedback": str}
3. 审核循环：
   - 通过（score >= 7）→ 返回结果
   - 不通过 → 带反馈重做（最多 3 轮）
   - 超过 3 轮 → 强制返回 + 警告
4. 函数签名：supervisor(task: str, max_retries: int = 3) -> dict
5. 返回值包含：output, attempts, final_score, warning(可选)
6. 包含 if __name__ == "__main__" 的测试入口

依赖：使用 workflows/model_client.py 的 chat() 函数
chat() 返回 (text, usage) 元组
```

生成的代码：（参考实现）

```
"""Supervisor 监督模式 — Worker 执行 + Supervisor 审核 + 反馈修正循环

核心设计：
- Worker 只负责执行，Supervisor 只负责审核（职责隔离）
- 审核反馈结构化（评分 + 弱项 + 建议），支持定向修改
- max_retries=3 兜底，避免无限循环
"""

import json

from workflows.model_client import chat

```

```

WORKER_SYSTEM = """你是 AI 技术分析师。请按要求完成分析任务。
输出 JSON 格式，包含：summary，key_points，recommendation。"""

SUPERVISOR_SYSTEM = """你是质量审核专家。请审核以下分析报告。

评分维度（每维度 1-10）：
1. 准确性：信息是否准确无误
2. 深度：分析是否有洞察力
3. 格式：是否符合 JSON 规范，结构清晰

输出严格 JSON：
{"passed": true/false, "score": 1-10总分, "feedback": "具体改进建议"}
只输出 JSON，不要其他内容。"""


def supervisor(task: str, max_retries: int = 3) -> dict:
    """监督模式：Worker 产出 + Supervisor 审核，不通过就重做

    Args:
        task: 分析任务描述
        max_retries: 最大重试次数

    Returns:
        {"output": str, "attempts": int, "final_score": int, "warning":
    """
    worker_output = None
    feedback = ""

    for attempt in range(1, max_retries + 1):
        # --- Worker 执行 ---
        if attempt == 1:
            worker_output, _ = chat(task, system=WORKER_SYSTEM)
        else:
            # 带反馈重做
            revision_prompt = (
                f"原始任务: {task}\n\n"
                f"上次产出: {worker_output}\n\n"
                f"审核反馈: {feedback}\n\n"
                f"请根据反馈改进，保持 JSON 格式。"
            )
            worker_output, _ = chat(revision_prompt, system=WORKER_SYSTE

        # --- Supervisor 审核 ---
        review_prompt = f"请审核以下分析报告：\n{worker_output}"
        review_text, _ = chat(review_prompt, system=SUPERVISOR_SYSTEM, t

        try:
```

```
            review_data = json.loads(review_text)
        except json.JSONDecodeError:
            review_data = {"passed": False, "score": 0, "feedback": "审核

        score = review_data.get("score", 0)
        feedback = review_data.get("feedback", "请改进质量")
        print(f"  第 {attempt} 轮审核: 得分 {score}/10")

        # --- 判断是否通过 ---
        if review_data.get("passed", False) or score >= 7:
            return {
                "output": worker_output,
                "attempts": attempt,
                "final_score": score,
            }

    # 达到最大重试次数
    return {
        "output": worker_output,
        "attempts": max_retries,
        "final_score": score,
        "warning": f"达到最大重试次数({max_retries})，可能质量不达标",
    }


# --- 测试入口 ---
if __name__ == "__main__":
    print("=" * 50)
    print("Supervisor 监督模式测试")
    print("=" * 50)

    result = supervisor("请分析 LangGraph 框架的优缺点和适用场景")

    print(f"\n最终结果:")
    print(f"  审核轮次: {result['attempts']}")
    print(f"  最终得分: {result['final_score']}/10")
    if result.get("warning"):
        print(f"  警告: {result['warning']}")
    print(f"  输出预览: {result['output'][:200]}...")
```

步骤 2：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

> `请解释 patterns/supervisor.py 的设计：`
>
> `1. 为什么 Worker 和 Supervisor 用不同的 system prompt？`

> `2. Supervisor 的 temperature=0.2 有什么作用？`
> `3. "带反馈重做"和"盲目重试"有什么区别？`
> `4. 为什么 max_retries=3 是合理的？`

关键设计解读：

设计点	为什么这样做
职责隔离	Worker 只执行，Supervisor 只审核，避免自己评自己
`temperature=0.2`	审核需要一致性，不需要创造力
结构化反馈	"score + feedback" 让 Worker 知道该改什么
强制返回	超过 3 轮不再重试，加 warning 标记

步骤 3：运行测试

```
cd ~/ai-knowledge-base
python patterns/supervisor.py
```

期望输出：

```
==================================================
Supervisor 监督模式测试
==================================================
  第 1 轮审核: 得分 6/10
  第 2 轮审核: 得分 8/10

最终结果:
  审核轮次: 2
  最终得分: 8/10
  输出预览: {"summary": "LangGraph 是基于有向图的..."}...
```

步骤 4：验证

检查清单：

检查项	期望	实际
第 1 轮审核有评分输出	是	
通过后立即返回（不多跑）	是	
结果包含 output + attempts + final_score	是	
不超过 3 轮审核	是	

步骤 5：提交到 Git

```
git add patterns/supervisor.py
git commit -m "feat: add Supervisor pattern with review loop (max 3 retr
```

完成！ Supervisor 模式实现了"执行 → 审核 → 定向修改 → 再审核"的质量闭环。最多 3 轮，避免无限循环。